Lab 13: Code Refiactoring – Improving Legacy Code with AI Suggestions

Course Title: AI Assisted Coding
Name:G.Varshith Reddy
Enrollment_no:2303a52287
Batch-37

Task 1: Refiactoring – Eliminating Repetitive Logic

Exact Task Description: Task: Use AI-assisted coding techniques to restructure a Python program that perfiorms repeated calculations, transfiorming it into a reusable and maintainable design.

Instructions:

Ask AI to locate repeated computations within the program.

Encapsulate repeated logic into reusable fiunctions.

Verifiy that program output remains unchanged after refiactoring.

Add proper docstrings to all defined fiunctions.

Prompt Used:

> "Identifiy the repeated area and perimeter calculations in this Python script. Please refiactor the code by encapsulating this logic into a single reusable fiunction with proper docstrings, and call it fior the provided dimensions to ensure the output remains exactly the same."

Refiactored Code:

```python
def print_geometry_details(length, width):
    """
    Calculates and prints the area and perimeter of a rectangle.

    Args:
        length (int/float): The length of the rectangle.
        width (int/float): The width of the rectangle.
    """
    area = length * width
    perimeter = 2 * (length + width)
    print("Area:", area)
    print("Perimeter:", perimeter)

# Using the reusable function for different inputs
print_geometry_details(8, 4)
print_geometry_details(12, 6)
```

Expected Output:

```
Area: 32
Perimeter: 24
Area: 72
Perimeter: 36
```

Explanation:

I created a reusable print_geometry_details fiunction that takes length and width as parameters. This eliminates the need to write the math operations over and over again, making the code much cleaner and easier to update ifi we ever need to change how the calculations are done.

Task 2: Refiactoring – Modularizing Inline Calculations

Exact Task Description:

Task: Use AI to refiactor a Python script by extracting repeated inline calculations into reusable and well-documented fiunctions.

Instructions:

Identifiy repeated calculation patterns in the code.

Convert them into reusable fiunctions.

Ensure readability and proper documentation.

Prompt Used:

> "Refiactor this Python script by extracting the repeated inline discount calculations into a modular, reusable, and well-documented fiunction. Then, run the fiunction fior the amounts 1000 and 2000."

Refiactored Code:

```python
def calculate_final_amount(amount, discount_rate=0.10):
    """
    Calculates the final amount after applying a discount.

    Args:
        amount (float): The original price/amount.
        discount_rate (float): The discount percentage (default is 10%).

    Returns:
        float: The final amount after the discount is subtracted.
    """
    discount = amount * discount_rate
    return amount - discount

# Applying the function to the provided amounts
print("Final Amount:", calculate_final_amount(1000))
print("Final Amount:", calculate_final_amount(2000))
```

Expected Output:

```
ects/AI Assistant Coding/Assignment-13.py"
Final Amount: 900.0
Final Amount: 1800.0
```

Explanation:

The inline discount logic was pulled out into a dedicated calculate_final_amount fiunction. By doing this, we remove the copy-pasted calculation steps firom the main flow, making the script easier to read and heavily reducing the chance ofi math errors in the fiuture.

Task 3: Refiactoring – Improving Loop and Conditional Perfiormance

Exact Task Description:

Task: Apply AI-based suggestions to enhance the perfiormance ofi Python code that relies on nested loops and repeated conditional checks.

Instructions:

Use AI to recommend more efficient data structures or logic simplifications.

Replace inefficient constructs while preserving program behavior.

Compare execution efficiency befiore and after refiactoring.

Prompt Used:

> "Optimize this Python code that uses inefficient nested loops fior lookups. Recommend a more efficient data structure (like a set) to eliminate the inner loop entirely, preserving the exact output behavior while improving execution speed."

Refiactored Code:

```python
import time

# Original lists
students = ["Anil", "Bhavya", "Charan", "Divya"]
check_list = ["Charan", "Esha", "Bhavya"]

# Start performance timer
start_time = time.perf_counter()

# AI Suggestion: Convert list to a set for O(1) instant lookups
students_set = set(students)

# Efficient single loop lookup
for name in check_list:
    if name in students_set:
        print(name, "found")
    else:
        print(name, "not found")

# End performance timer
end_time = time.perf_counter()
print(f"\n[Execution completed in {end_time - start_time:.6f} seconds]")
```

Expected Output:

```
ects/AI Assistant Coding/Assignment-13.py"
Charan found
Esha not found
Bhavya found

[Execution completed in 0.000230 seconds]
```

Explanation:

I replaced the slow nested fior loop with a Python set fior the students list, which allows fior instant lookups instead ofi scanning the whole list every time. This dramatically speeds up execution time while keeping the program's output exactly the same.

Task 4: Refiactoring – Replacing Hardcoded Values with Constants

Exact Task Description:

Task: Utilize AI assistance to improve maintainability by replacing hardcoded numerical values with named constants.

Instructions:

Detect hardcoded numeric values used directly in expressions.

Define descriptive constants at the beginning ofi the program.

Update calculations to use these constants without changing results.

Prompt Used:

> "Improve the maintainability ofi this Python code by identifiying the hardcoded 'rate' and 'time' numbers in the simple interest calculations. Replace them with descriptive named constants defined at the top ofi the file."

Refiactored Code:

```python
# Descriptive constants defined at the beginning of the program
TIME_IN_YEARS = 2
INTEREST_RATE = 5

# Calculations updated to use constants instead of hardcoded 'magic numbers'
print("Simple Interest:", (1000 * TIME_IN_YEARS * INTEREST_RATE) / 100)
print("Simple Interest:", (2000 * TIME_IN_YEARS * INTEREST_RATE) / 100)
```

Expected Output:

```
ects/AI Assistant Coding/Assignment-13.py"
Simple Interest: 100.0
Simple Interest: 200.0
```

Explanation:

I extracted the literal numbers 2 and 5 into named, uppercase constants TIME_IN_YEARS and INTEREST_RATE at the top ofi the script. This gives the numbers actual context so anyone

reading the fiormula understands what it means, and it allows fior global updates with just one edit.

Task 5: Refiactoring – Enhancing Variable Naming and Code Clarity

Exact Task Description:

Task: Apply AI tools to improve code readability by renaming unclear variables and adding meaningfiul comments.

Instructions:

Replace non-descriptive variable names with meaningfiul identifiers.

Add inline comments where the logic may be unclear.

Ensure program output remains the same.

Prompt Used:

> "Refiactor this Python snippet to improve clarity. Replace the non-descriptive generic variables (x, y, z) with meaningfiul identifiers representing a shopping calculation, and add inline comments explaining each step."

Refiactored Code:

```python
# Define the base price of the item
base_price = 50

# Define the applicable tax amount
tax_amount = 30

# Calculate the final total cost by adding base price and tax
total_cost = base_price + tax_amount

# Display the final calculated cost to the user
print(total_cost)
```

Expected Output:

```
ects/AI Assistant Coding/Assignment-13.py"
80
```

Explanation:

I replaced the vague variables x, y, and z with descriptive names like base_price and total_cost to give the numbers a real-world shopping context. I also added inline comments to clearly outline exactly what each line ofi the script is trying to accomplish.